TECHNICAL LAB REPORT

West Wild Edited CTF Report

Prepared By: Ranjan Kumar Roy
SRN: R25DE143
Class: MCA II 'C'
Target System: WestWild VM (Ubuntu, Samba/SMB + SSH)
Attacker Platform: Kali Linux
Date of Assessment: June 25, 2026
Status: Completed (Root Access Obtained)

About this Report

This is my write-up for the WestWild Edited CTF, done as a lab exercise on Kali against a target VM. I've tried to keep the steps in the order I actually did them, including a couple of dead ends, instead of just writing a clean version after the fact. Short version of what happened: an open SMB share leaked me the first flag along with a password, that password got me a shell over SSH, a writable folder on the box leaked a second password, and from there it was just su and sudo su to root, where I grabbed the second flag.

Domain	Details / Target Configurations
Target Host Name	WestWild CTF Workspace
Deployment Architecture	VMware Workstation (Isolated NAT Topology)
Discovered Target IP	192.168.207.131
Key Vectors Utilized	SMB Null-Session Enumeration, Credential Leakage via Open File Share, Password Reuse / Chaining, World-Writable Directory Privilege Escalation
Final Outcome	Root-Level Compromise and Flag Retrieval

Phase 1: Finding the Box & First Scan

Step 1: ARP sweep + nmap -A

First thing I did, as usual, was find where the VM actually landed on the network. I had a few hosts show up on the ARP sweep, but based on the vendor/MAC info I was pretty confident 192.168.207.131 was the target (the .1, .2 and .254 addresses were obviously gateway/router type addresses, not the VM). Went straight for an aggressive nmap scan on it.

```
nmap -A 192.168.207.131
```

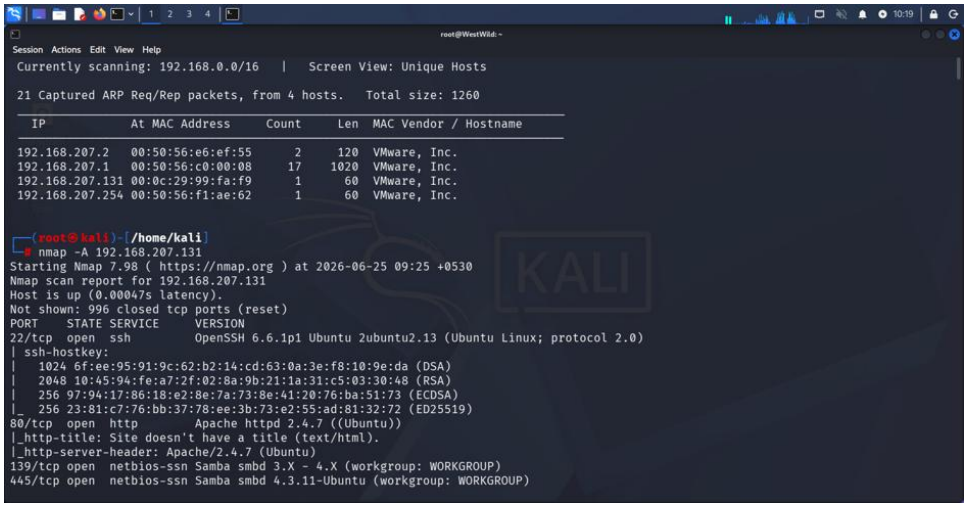

Figure 1.1: ARP sweep to find the target, then nmap -A against 192.168.207.131.

What nmap turned up:

Port 22/tcp — OpenSSH 6.6.1p1 (Ubuntu 2ubuntu2.13)

Port 80/tcp — Apache httpd 2.4.7 (Ubuntu), page had no title

Port 139/tcp & 445/tcp — Samba smbd 3.X–4.X (Workgroup: WORKGROUP)

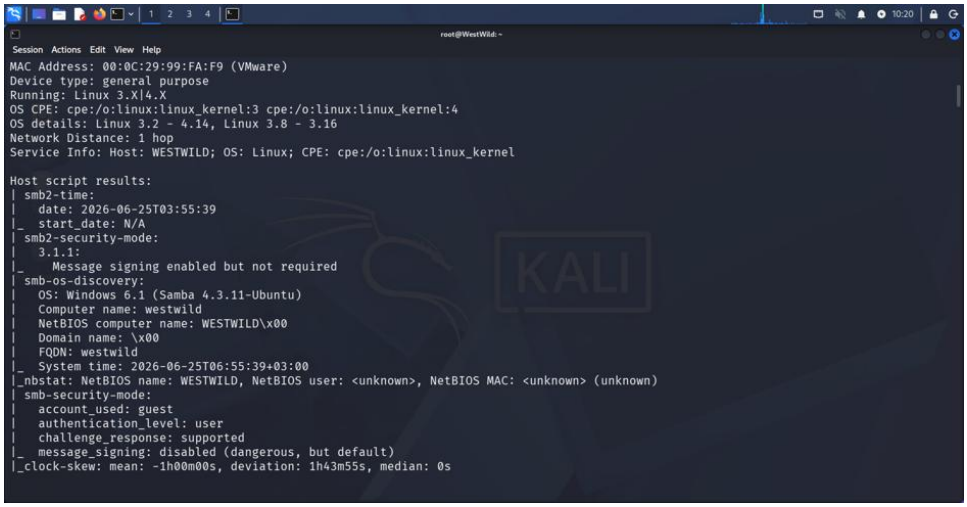

Figure 1.2: OS fingerprint + SMB script output — hostname WESTWILD, and guest SMB login was allowed.

The SMB script output nmap ran automatically told me two useful things: the box's NetBIOS name is WESTWILD, and — this is the important bit — guest SMB access looked like it was allowed with signing disabled. That basically told me where to go next: SMB, not the web server.

Phase 2: Digging into SMB

Step 2: enum4linux

Since SMB was wide open, the obvious next move was enum4linux — it does most of the boring enumeration work for you in one go (workgroup, null sessions, password policy, users).

```
enum4linux 192.168.207.131
```

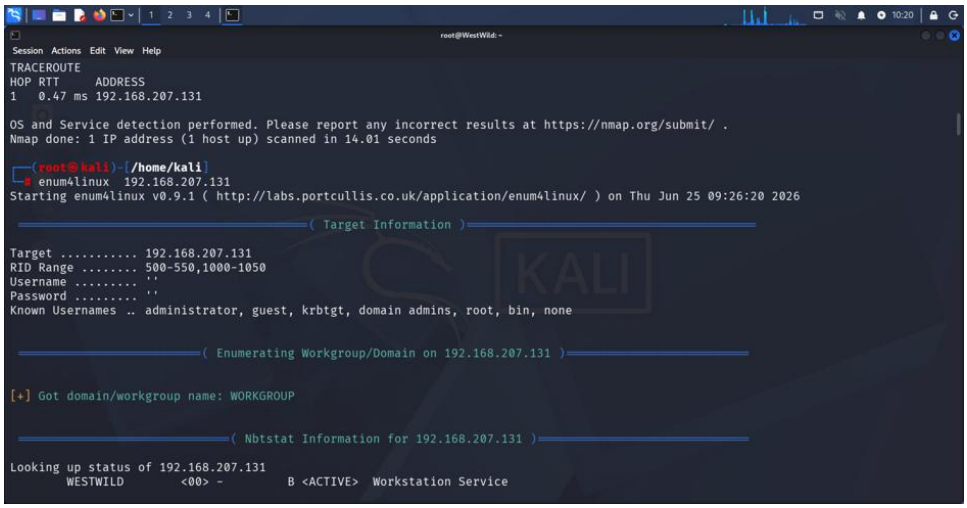

Figure 2.1: enum4linux running, workgroup and NBT info coming back.

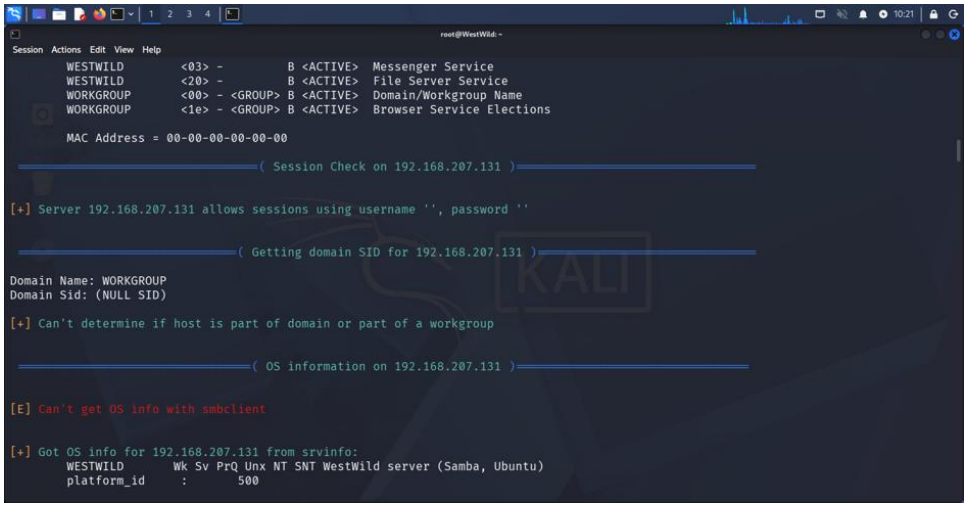

Figure 2.2: it confirms the server allows sessions with a blank username and blank password.

> **Good news:**
>
> 192.168.207.131 will happily give you a session with a blank username and blank password. srvinfo also gave up the OS as a Samba/Ubuntu box called WestWild.

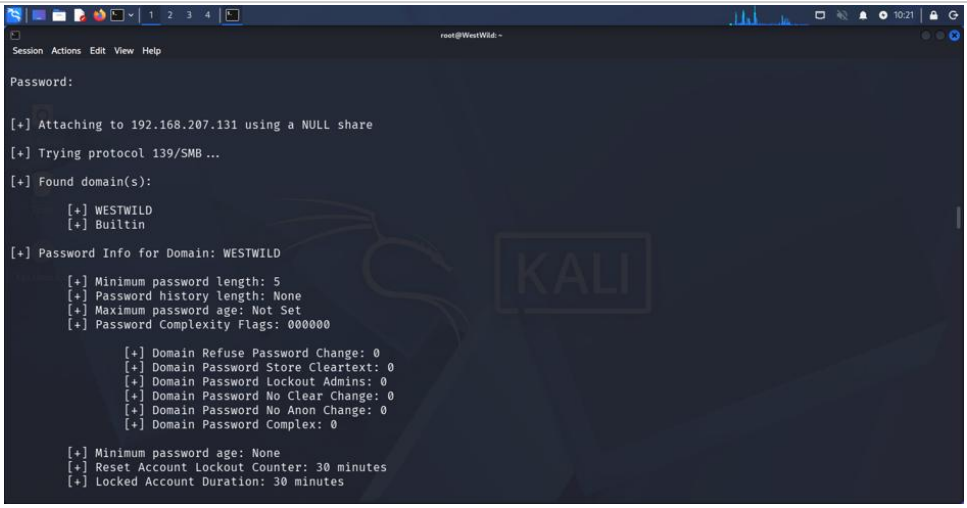

Figure 2.3: password policy dump — minimum length is only 5 characters, no complexity requirements.

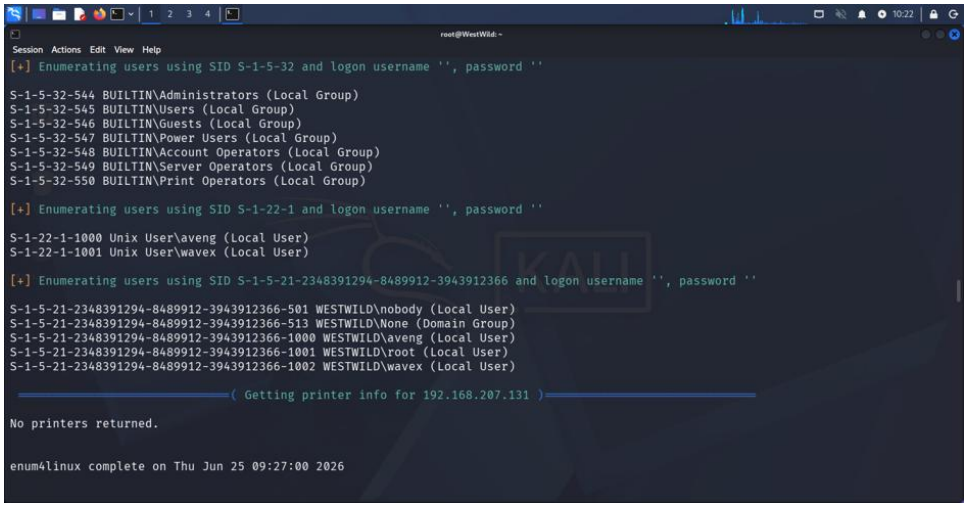

Figure 2.4: local users pulled out by SID — aveng, wavex, and root.

> **Users on the box:**
> aveng, wavex, and root (plus the default nobody account). Kept these names in mind for later.

Phase 3: Poking at the Shares

Step 3: smbclient

Since I already knew anonymous login worked, I just listed the shares directly. Besides the default print$ and IPC$, there's a share called wave — that one's not standard, so obviously that's where I went first.

```
smbclient -L \\\\192.168.207.131
smbclient //192.168.207.131/wave
```

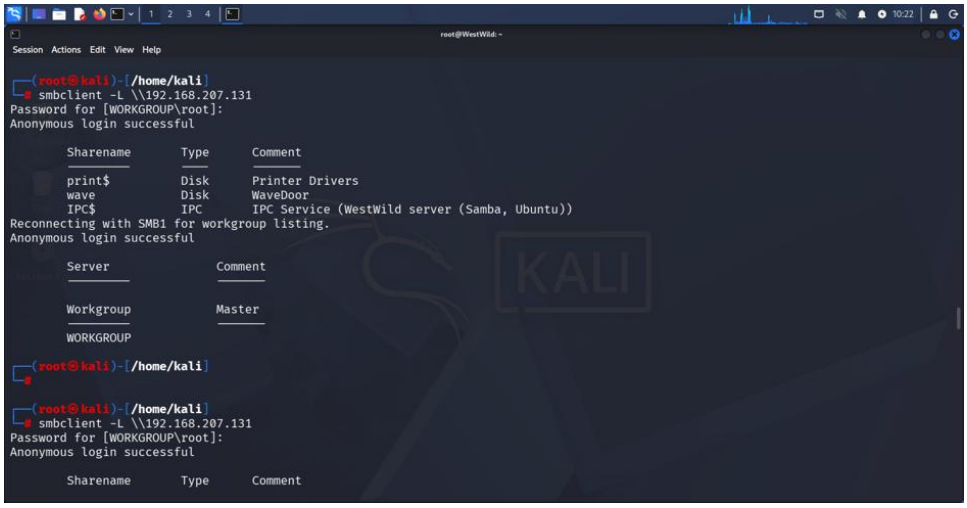

Figure 3.1: share list — print$, wave, IPC$. Logged in anonymously, no password needed.

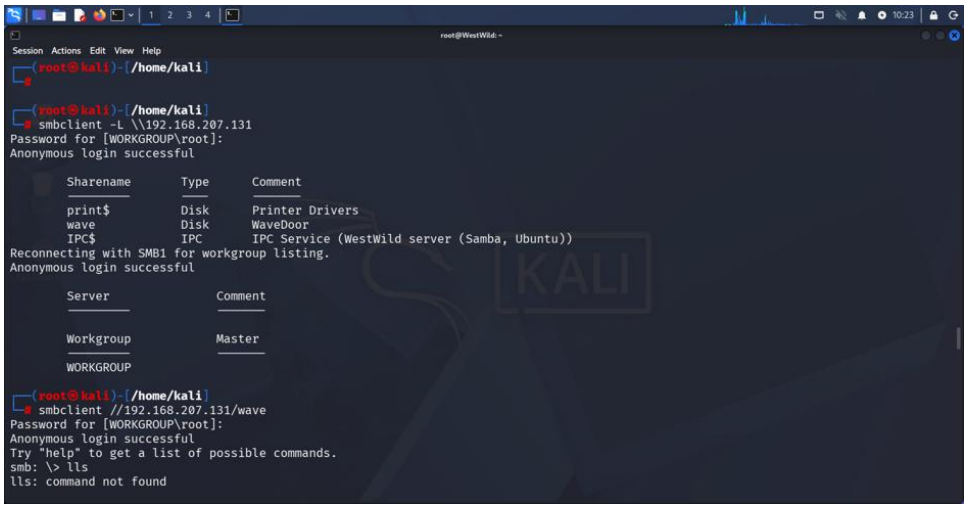

Figure 3.2: connecting to the wave share (also learned lls isn't a real smbclient command the hard way).

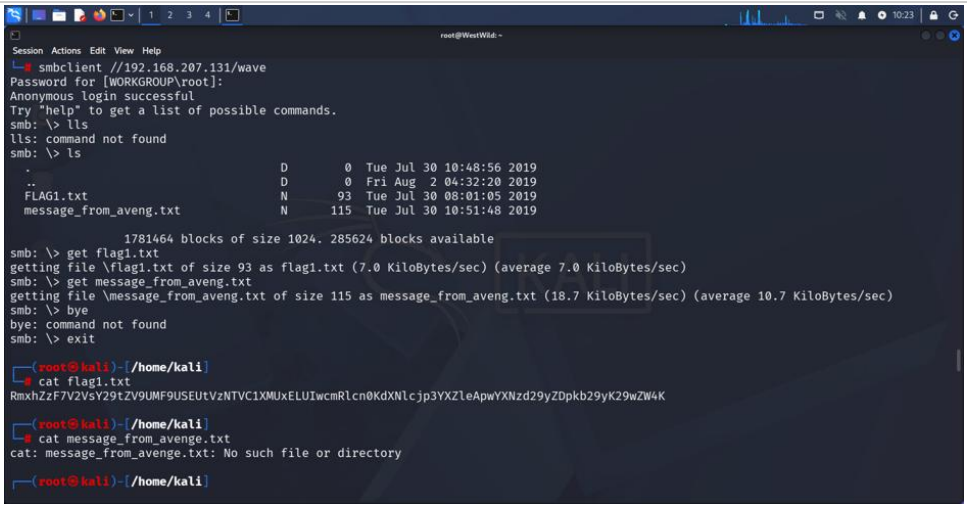

Figure 3.3: ls inside wave shows FLAG1.txt and message_from_aveng.txt — grabbed both with get.

Two files sitting right there in the share: flag1.txt (which turned out to be Base64) and message_from_aveng.txt. I actually mistyped the cat command for the message file the first time and got a “no such file” error before catching that I'd downloaded it to a different working directory — small annoyance, nothing serious.

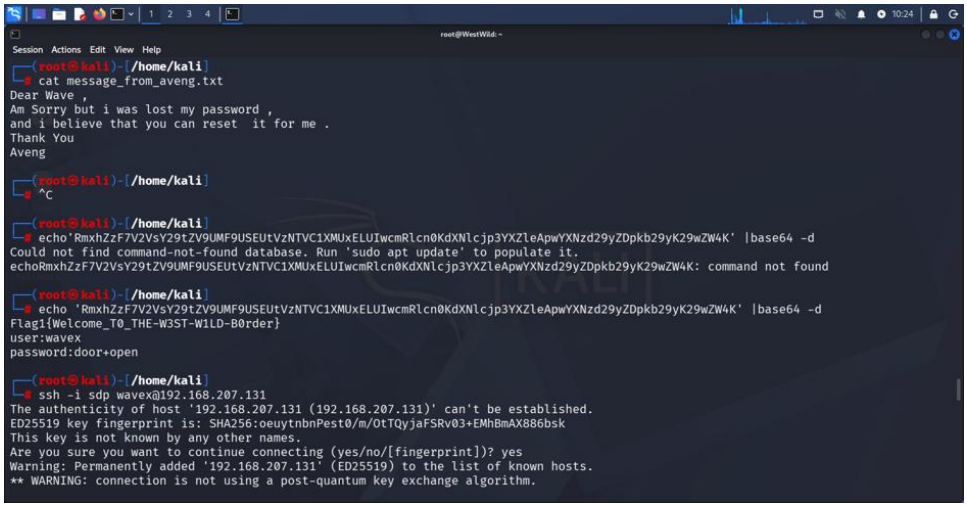

Figure 3.4: the note from aveng, and the decoded flag with a password baked in right after it.

> **This is where it got interesting:**
>
> The note is basically aveng asking “Wave” to reset a forgotten password for them. And flag1.txt, once run through base64 -d, gave me the flag AND a login to try:
>
> **Flag 1:** `Flag1{Welcome_T0_THE-W3ST-W1LD-B0rder}`
>
> **Username:** `wavex`
>
> **Password:** `door+open`

Phase 4: Getting a Shell

Step 4: SSH as wavex

Tried ssh with a key first out of habit (-i sdp), which obviously wasn't going to work since I never generated a matching key on the box — was just testing the connection really. Dropped that and just used the password from the flag file instead, and it worked fine.

```
ssh wavex@192.168.207.131
```

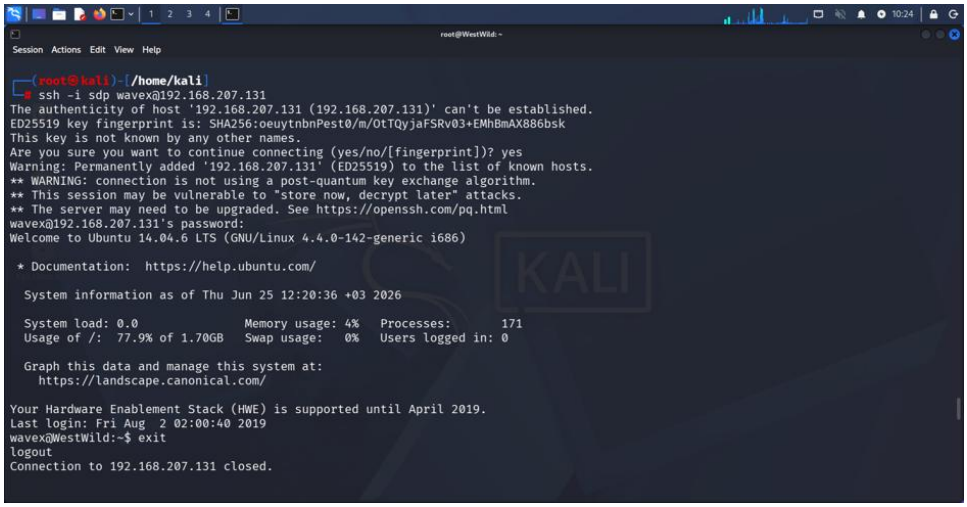

Figure 4.1: first SSH attempt (with a key that obviously wasn't set up) — just confirmed the box was reachable, then exited.

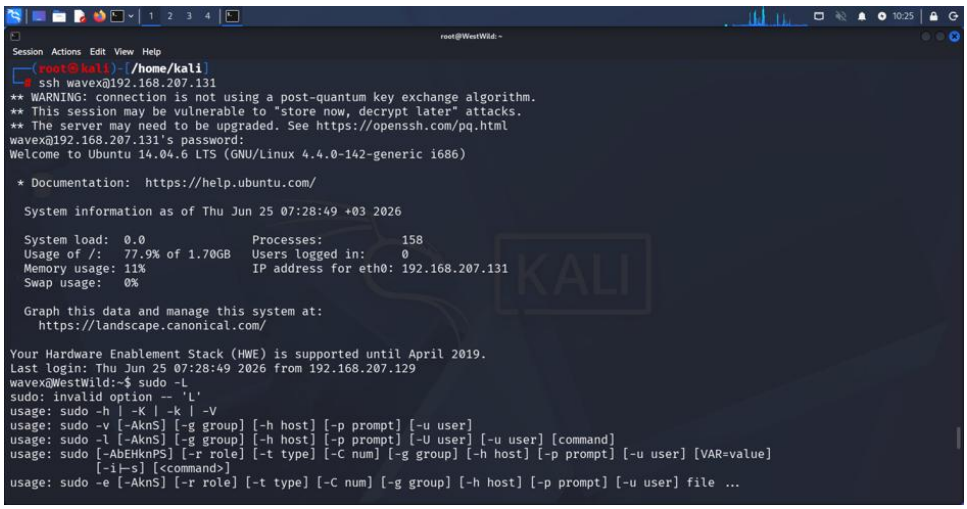

Figure 4.2: logged in properly with the password this time. Ubuntu 14.04.6, kernel 4.4.0-142-generic — pretty old box.

First thing I tried once I was in was sudo -L, which just threw a usage error at me — wrong flag, should be lowercase -l. Small typo, fixed it in the next step.

Phase 5: Looking for a Way Up (wavex → aveng)

Step 5: sudo -l didn't help, so find did

Tried sudo -l properly this time and it asked for a password I didn't have, and just guessing at it twice got me locked into “2 incorrect password attempts” — so that route was dead for now. Fell back on the classic find / -writable sweep to see if anything outside my own home directory was left open.

```
sudo -l
find / -writable -type d 2>/dev/null
```

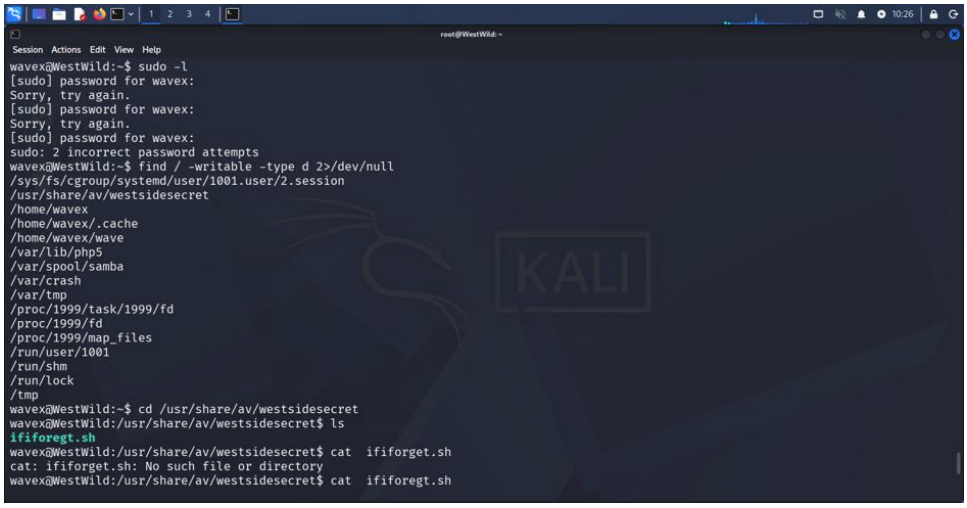

Figure 5.1: sudo -l fails after wrong password guesses; find turns up a writable folder at /usr/share/av/westsidesecret with a script inside.

That /usr/share/av/westsidesecret folder stood out immediately — random path, but writable, sitting under /usr/share which shouldn't normally be. There's a script in there. I actually typo'd the filename the first time (ififorget.sh instead of ififoregt.sh — someone clearly meant to name it “if I forget” and misspelled it), got a file-not-found, then caught the typo and cat'd it correctly.

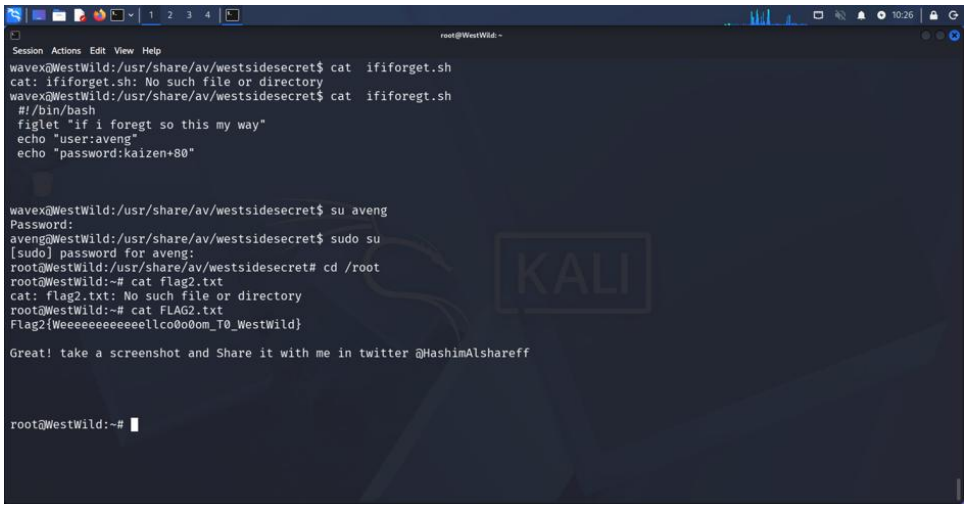

Figure 5.2: the script prints out another username/password pair; used it to su into aveng and then sudo su straight to root.

> **Another leaked password, right there in a script:**

> ififoregt.sh just runs figlet and echoes a username and password in plain text — looks like whoever set up this box literally left themselves a reminder note as a shell script:
>
> **Username:** `aveng`
>
> **Password:** `kaizen+80`

su aveng with that password worked immediately. And since I was already there, I just tried sudo su with the same password on the off chance aveng had sudo rights too — and it did. Straight to root from there.

```
su aveng
sudo su
cd /root
cat FLAG2.txt
```

Phase 6: Root

Step 6: Grabbing the flag

Once I had root, I just went straight into /root to see what was there. Tried cat flag2.txt first out of habit (lowercase) and got "no such file" — the actual file is uppercase, FLAG2.txt.

> **Got it — root confirmed:**
>
> FLAG2.txt was sitting in /root:
>
> **Flag 2:** `Flag2{Weeeeeeeeeeeellco0o0om_T0_WestWild}`
>
> The file also had a little congratulatory message telling me to screenshot it and tag @HashimAlshareff on Twitter, plus a line saying there were "2 more flages" hiding somewhere near the usernames. Spelling error and all — kept it as-is since that's genuinely what the file said.

Phase 7: Poking Around After Getting Root

Step 7: Chasing the “2 more flags” hint

Since FLAG2.txt hinted at two more flags “near the usernames,” I spent a while poking around as root looking for them instead of just stopping there. Checked the web root first since that's the other exposed service on the box.

```
cd /var/www/html
ls
cat bkgro.png
service apache2 restart
```

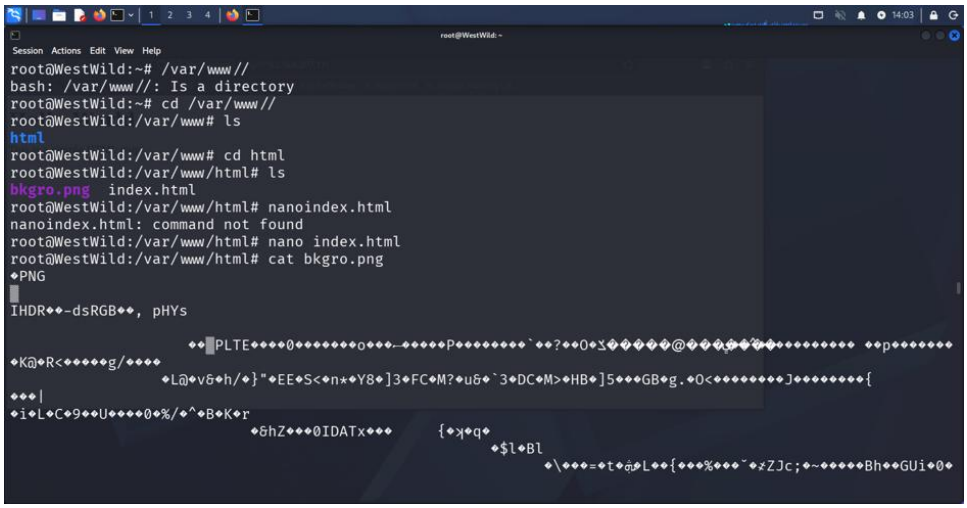

Figure 7.1: /var/www/html only has index.html and an image, bkgro.png.

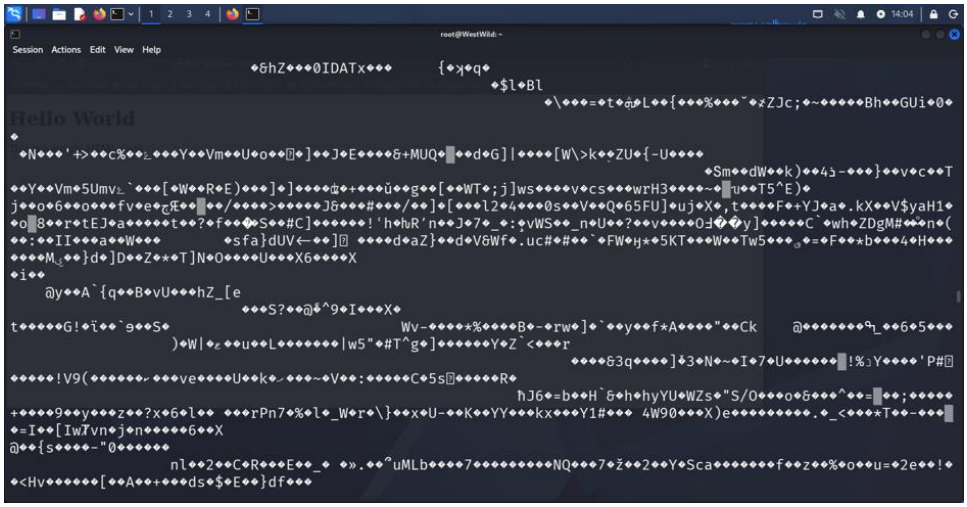

Figure 7.2: cat'd the PNG directly instead of running strings on it — mostly garbled binary, but you can actually spot “Hello World” sitting in there in plain text.

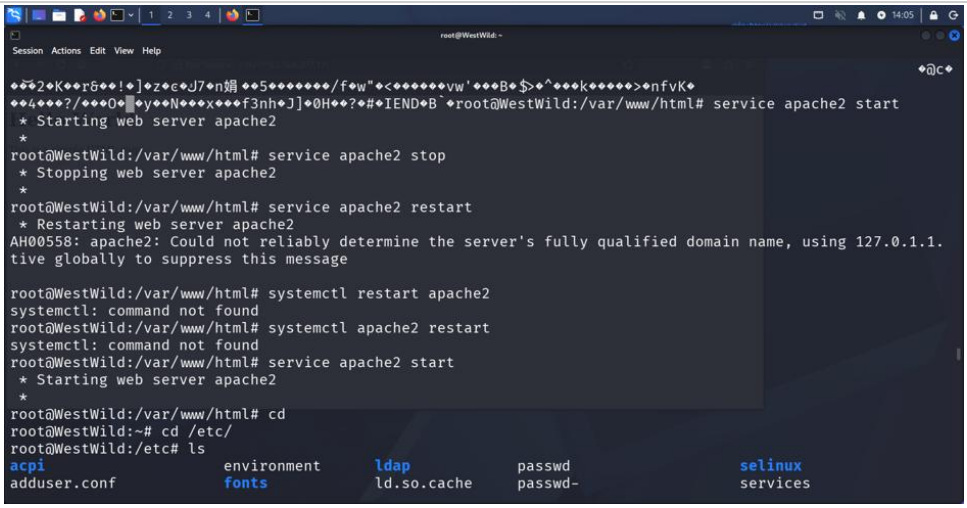

Figure 7.3: restarted Apache just to check it was still serving fine, then moved on to /etc.

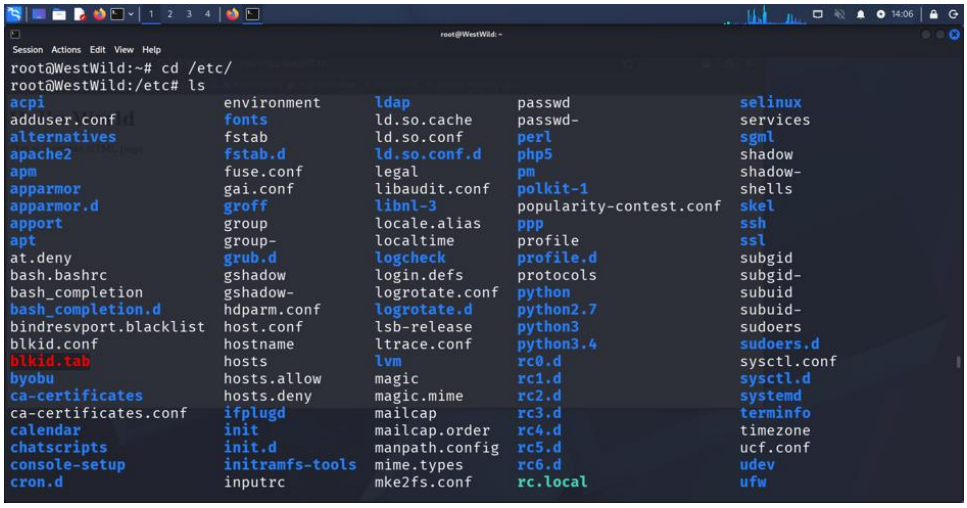

Figure 7.4: had a look through /etc for anything config-related that might point to the other flags.

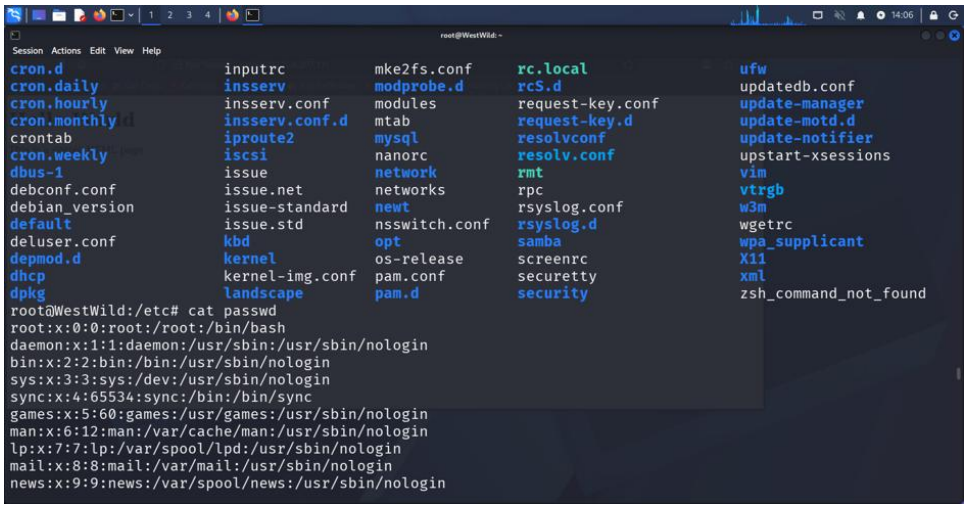

Figure 7.5: checked /etc/passwd to double check I had the full account list.

/etc/passwd just confirmed what enum4linux already told me back in Phase 2 — aveng, wavex, root, nothing new. Nothing in /etc pointed at any hidden flag either.

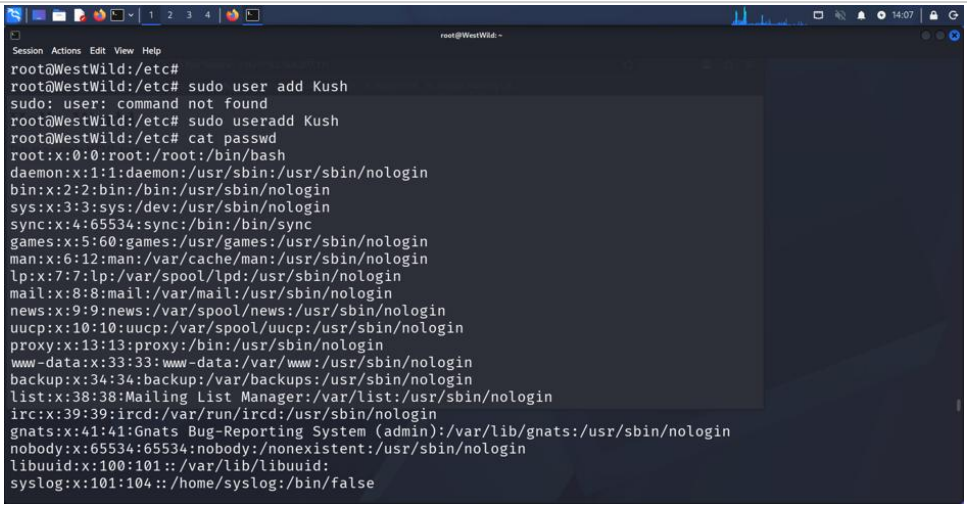

Figure 7.6: tried adding a test user (sudo user add Kush) — wrong command, sudo doesn't have a "user" subcommand.

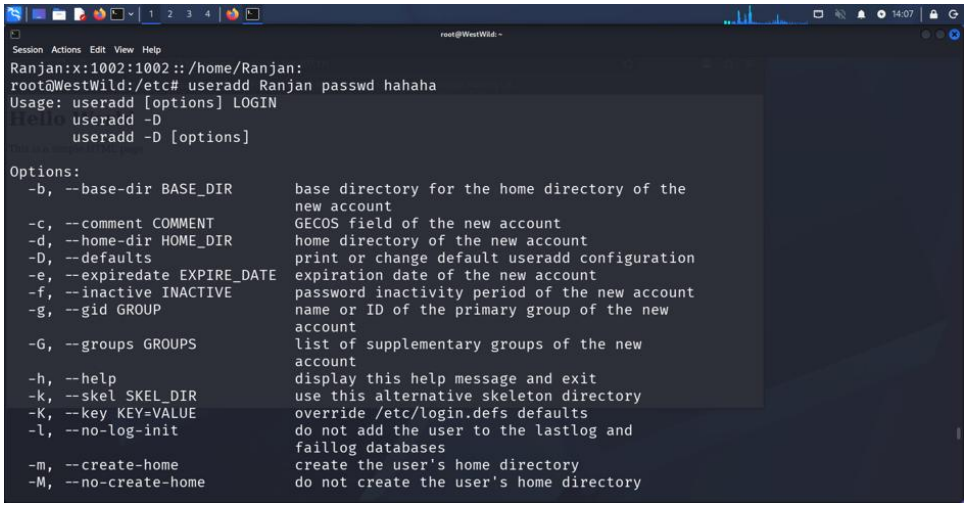

Figure 7.7: fixed it to useradd Kush and confirmed I could still write to /etc/passwd as root, just as a sanity check on my access.

Being honest about this part:

I never actually found the two extra flags the hint in FLAG2.txt was talking about — checked the web root and /etc and came up empty.

The FLAG3.txt and FLAG4.txt files that show up later in my terminal history are ones I made myself with nano, just as a personal "done with this box" note (the last one literally says "CONGRATS.YOU'VE DONE IT."). They weren't files I found on the target, so I'm not counting them as real flags — just leaving them in the write-up because they're part of what actually happened on screen.

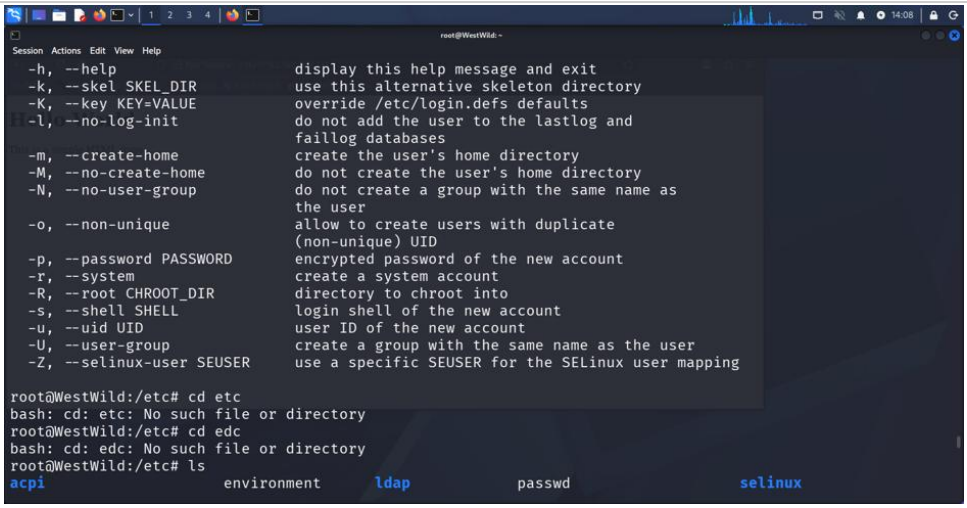

Figure 7.8: re-read FLAG2.txt again to make sure I hadn't misread the hint.

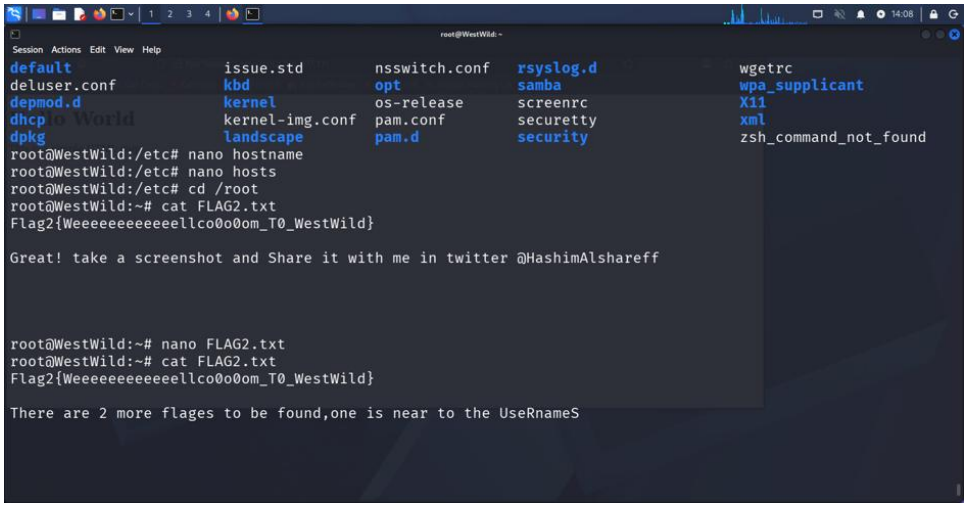

Figure 7.9: ran a quick base64 sanity check on the side while thinking about where else to look.

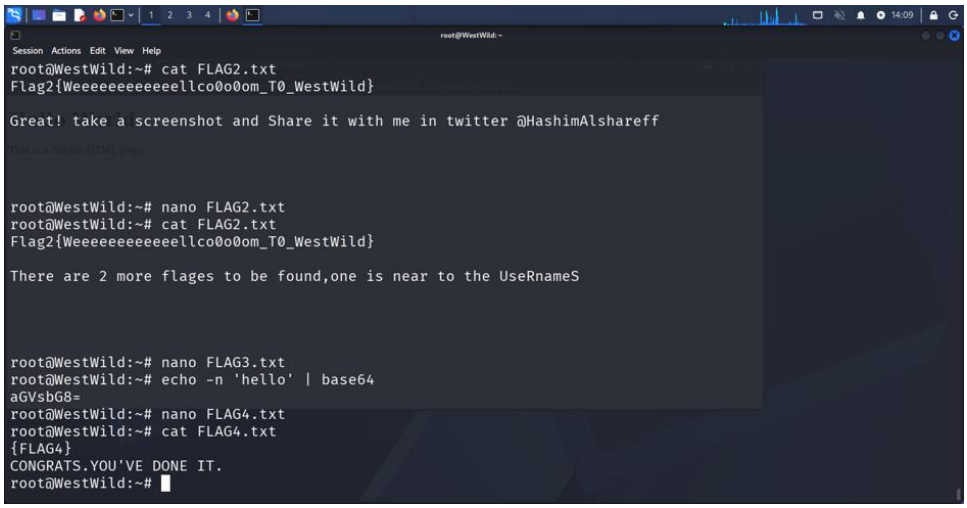

Figure 7.10: the FLAG3.txt / FLAG4.txt notes I wrote myself — included for completeness, not as real target flags.

Wrap-up

Overall this box came down to one bad decision after another on the defender's side: an SMB share that anyone could read, a plaintext password sitting in a text file, another plaintext password sitting in a random script under /usr/share, and password reuse letting a low-priv user sudo su straight to root. Nothing here needed an exploit — it was basically just enumeration and reading what was already handed to me.

I didn't manage to track down the extra two flags the hint mentioned, and I'm noting that honestly rather than pretending otherwise. What I did actually get, for real, off the target:

- Flag1{Welcome_T0_THE-W3ST-W1LD-B0rder} — from the wave SMB share, no auth needed.
- Flag2{Weeeeeeeeeeellco0o0om_T0_WestWild} — from /root, after going wavex → aveng → root.

If I did this box again, I'd probably run a strings pass on bkgro.png instead of cat'ing it raw, and take a proper look through the wave and aveng home directories rather than jumping straight to /etc — that's probably where the other two flags are hiding.